Re^{2}po: A Distributed Redundancy-aware Model Repository in Edge Serverless Computing

Sheng Wang*, Haohui Zheng*, Hongmin Geng, Jie Zhang, Lin Gu, Deze Zeng†, *Senior Member, IEEE*

***Abstract*—Integrating the emerging serverless paradigm into edge computing could provide a highly efficient serving mode for edge applications, e.g., optimizing Deep Neural Network (DNN) inference in an on-demand manner. Despite its benefits, current serverless inference rely on retrieving DNN models from centralized cloud repositories, which introduces latency, increases data transmission overhead, and retains redundant parameter data in models with similar functions. To overcome these inefficiencies, we propose Re^{2}po, a novel redundancy-aware distributed edge DNN model repository framework. Re^{2}po decomposes DNN models into individual layers, identifies and removes redundant ones, and stores only the unique layers. By positioning the repository near the edge serverless platform, Re^{2}po enables fast model retrieval, allowing serverless functions to efficiently download models and perform inference. Unlike existing model compression methods such as pruning and quantification, Re^{2}po preserves inference accuracy by fully reconstructing the original model while significantly reducing storage requirements through redundancy removal. Experimental results indicate that Re^{2}po outperforms state-of-the-art chunk-based model storage methods, achieving improvements of 15.7% in upload time, 34.1% in download time, and 26.2% in storage efficiency.**

***Index Terms*—DNN inference, serverless, Redundancy-aware, Model Storage.**

I. Introduction

Serverless computing has gained significant popularity due to its ease of use, automatic scaling, pay-as-you-go pricing schemes, and freedom from infrastructure management [1]. As the era of intelligence advances, deploying intelligent applications within the serverless computing paradigm has been successfully applied across various domains with notable case studies in autonomous driving and smart home technologies. At the same time, Deep Neural Networks (DNNs) have emerged as the foundational backbone of these intelligent applications, thanks to their robust data processing capabilities afforded by inference processes.

Currently, the majority of serverless deployment strategies are primarily built on centralized cloud service architectures. While these cloud-centric serverless services depend on internet connectivity to interact with users or devices, they often suffer from higher communication latencies and increased resource consumption. To address these challenges, the edge serverless computing has emerged as a promising alternative. By shifting data processing tasks closer to the source, edge serverless computing can substantially reduce latency and optimize resource utilization. Consequently, edge serverless inference has become a prevalent approach offering robust, real-time inference capabilities for a wide range of intelligent applications.

Edge serverless inference services typically require the initialization of runtime functions and the downloading of necessary DNN models before execution [2]. These DNN models are often stored in cloud-based repositories, such as Hugging Face and PyTorch Hub. However, the significant geographical separation between serverless platforms and these cloud-hosted model repositories leads to considerable model download times for inference services. To validate this issue, we conduct several preliminary experiments on Function Compute, Alibaba serverless platform. Based on the function runtime logs, the average download speed is approximately 1.05 MB/s, requiring 55 seconds to download the 54.7 MB DenseNet-169 model from PyTorch Hub. As the size of DNN models continues to grow, the download latency becomes increasingly problematic. In edge environments with constrained network conditions, the download speed is even slower compared to that of public platforms. Hence, reducing the latency in downloading models for edge serverless inference has become a critical problem.

Building upon the challenges of downloading DNN models for edge serverless inference, it is important to understand the structure and nature of these models. DNN models are fundamentally composed of a series of layers, each comprising various tensors such as inputs, weights, and biases. The execution of inference tasks is achieved through the sequential computation of these layers. In response to model download latency, we conduct a detailed analysis of the architecture and data components of DNN model files, focusing on optimizing the transmission efficiency, particularly in network-constrained environments. Our analysis reveals that DNN models designed for similar functions often include numerous layers with identical tensor, which indicates the parameter and structure of these layers are exactly the same. Without loss of generality, we term these duplicate layers as 'redundant layers'. Consequently, the presence of such redundant layers significantly increases the data transmission overhead, and exacerbates the response time for edge serverless inference.

†Deze Zeng is the corresponding author (e-mail: deze@cug.edu.cn).

Sheng Wang, Haohui Zheng, Hongmin Geng and Deze Zeng are with China University of Geosciences, Wuhan, and the Engineering Research Center of Natural Resource Information Management and Digital Twin Engineering Software, Ministry of Education and School of Computer Science, Wuhan 430074, China (e-mail: {wangsheng124, 20201003309, hmgeng, deze}@cug.edu.cn)

Jie Zhang is with the Department of Computer Science and Engineering, The Hong Kong University of Science and Technology, Hong Kong SAR. (e-mail: csejzhang@ust.hk).

Lin Gu is with the School of Computer Science and Technology, Huazhong University of Science and Technology, China (e-mail: lingu@hust.edu.cn).

*These authors contribute equally to this work.

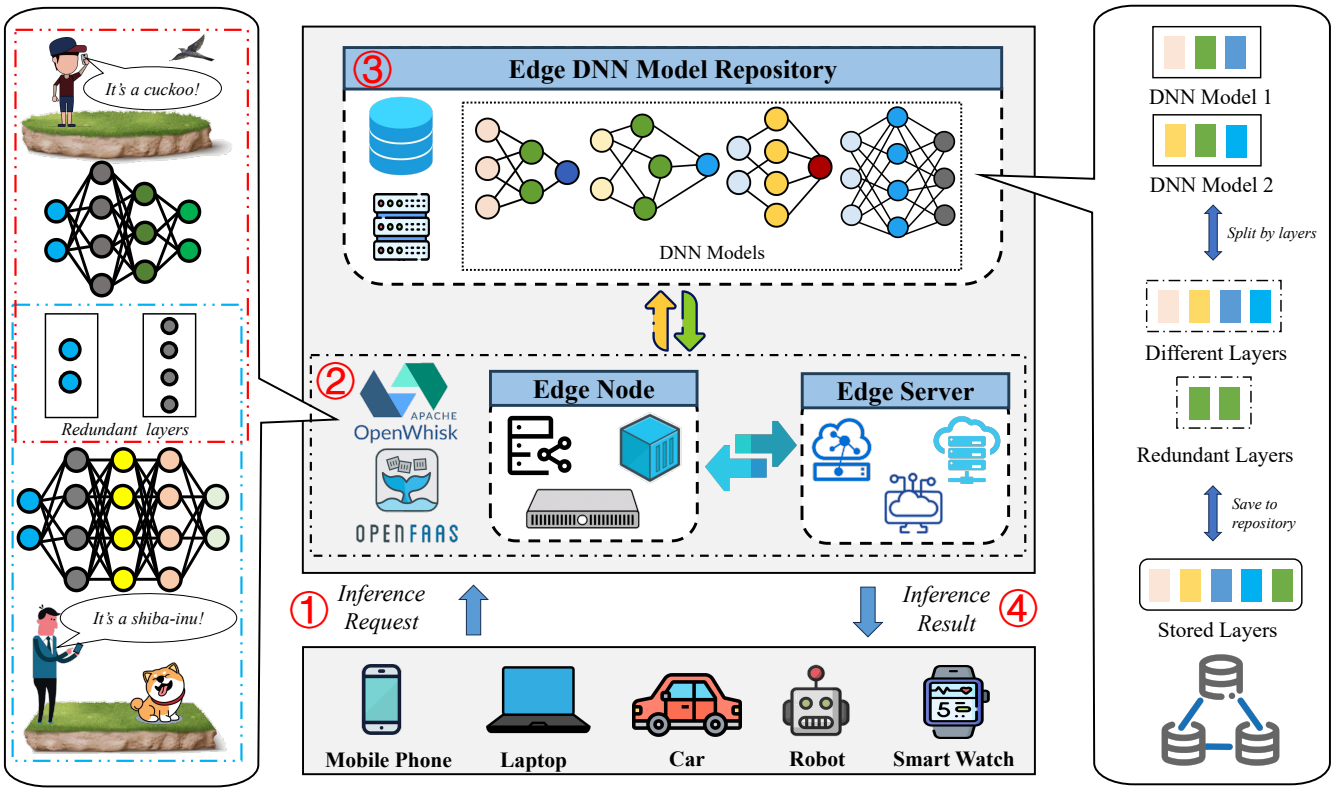


Fig. 1. A Scenario of Edge Serverless Inference for Intelligence Application.

To mitigate the transmission overhead caused by redundant layer, we propose minimizing redundant layers in models and constructing a novel layer-wise edge DNN model repository. Fig. 1 illustrates the scenario of edge serverless inference utilizing such repository framework. Serverless platforms deployed at the edge, e.g., OpenWhisk and OpenFaaS, directly send requests to the edge model repository to download models. Then, the repository reconstructs the complete model and returns it to the serverless platform, which finally performs inference and returns the results. By removing redundant layers during model storage, the repository reduces model size and improves storage efficiency, enabling it to store a greater number of models required for intelligent applications. For instance, in image classification tasks, lightweight models like EfficientNet-b0 can be used for low-accuracy, fast-response requests, while high-accuracy requests require larger models capable of handling more complex tasks, such as EfficientNet-b6. These models often share similar layers for input processing and may include identical layers for other data processing tasks. Removing these common redundant layers reduces model size, allowing the edge repository to store a wider variety of task-specific and precision-oriented models. At the same time, it also lowers transmission latency during downloading. It is worth noting that removing redundant layer differs fundamentally from techniques like model pruning or compression. While pruning and similar operations alter the model structure and can impact accuracy, removing redundant layers does not affect model accuracy, as the original model can be fully reconstructed. This distinction ensures that optimization through redundancy removal maintains the integrity and performance of the model.

To realize this concept, we design and implement Re^{2}po, a distributed redundancy-aware model repository framework that provides efficient and flexible model access while supporting edge serverless inference for intelligent applications. Specifically, Re^{2}po establishes a distributed model repository where models are decomposed into individual layers. Each layer is hashed to generate a unique identifier, enabling the identification and removal of redundant layers. By retaining only unique layers, Re^{2}po significantly improves the storage efficiency of the repository. As the edge model repository stores the necessary models for intelligent applications, serverless functions can efficiently download them on demand. This approach minimizes model access latency and accelerates edge serverless DNN inference, making Re^{2}po a powerful solution for optimizing storage and performance in edge environments.

Overall, the main contributions of our work can be summarized as:

- Through analyzing the structure and data of DNN models, we discover the layer redundancy problem in DNN models during edge intelligence application inference.
- We implement a prototype framework Re^{2}po, supporting for removing redundant layers and efficiently accessing DNN models.
- We extensively evaluate Re^{2}po with a set of benchmarks. Experimental results show that, among the tested models, Re^{2}po achieves a 15.7% reduction in average upload time, a 34.1% reduction in average download time, and a 26.2%

improvement in average storage efficiency compared to chunk-based model storage method.

In the rest of this paper, we first offer an overview of recent storage solutions for model repositories. Next, we conduct experiments and analyze the layer redundancy in existing models. Then, we elaborate on the distributed DNN model architecture based on layer-by-layer storage, which enhances both storage and model download efficiency. Finally, we present several open research challenges and explore potential solutions.

II. Related Work

Numerous studies have proposed various techniques to accelerate model loading on edge nodes. For instance, some research focuses on reducing model loading time by optimizing resource scheduling [3] [4] [5], caching mechanisms [6] [7], and other methods to shorten the process from loading to execution. However, these approaches primarily emphasize memory loading optimization, without fully addressing the time required to download models from the cloud, which is a process vulnerable to network fluctuations.

In addition to explore reducing loading latency, some efforts mainly focus on optimizing model storage size, which primarily emphasize reducing model precision cautiously to minimize performance degradation while decreasing storage and computational overhead of the model. Liang *et al.* [8] conduct a comprehensive survey on model pruning and quantization techniques. The survey provides practical guidelines for applying these compression methods, offering a broad perspective on the efficient deployment of DNN models. Kim *et al.* [9] propose a novel model compression method called PQK, which leverages pruning, quantization, and knowledge distillation to deploy deep neural networks on resource-constrained edge devices. Although PQK reduces the model size, adding the unused weights lead to performance degradation. To further reduce model parameters and decrease storage requirements, Xue *et al.* [10] propose a multi-layer sparse tensor decomposition method named as MLSTD, which effectively captures the intrinsic structural information within tensors by introducing multi-level sparsity representation and low-rank structure. However, MLSTD increases the complexity of the model due to setting different hyperparameters. Although those methods reduce the data size of DNN models, they inevitably reduce the accuracy of the DNN inference results, which is unacceptable for precision-sensitive applications.

However, current studies pay limited attention to efficient model access management and deduplicated storage mechanisms in edge environments. Wang *et al.* [11] introduce HDFS to optimize cloud computing data access storage algorithms, utilizing hash values to enhance the data access architecture and accelerate data access efficiency. For network edge devices, Li *et al.* [12] optimize the utilization of storage resources based on data similarity and popularity. However, this optimization only applies to hot data. Overall, existing distributed storage solutions do not account for the redundancy in DNN models, leading to a certain degree of storage resource waste. Reusing DNN models are crucial strategies to enhance storage efficiency. Madhyastha *et al.* [13] construct a lightweight and scalable model repository called DStore using finer-grained tensors. But DStore has limited support for concurrent access by multiple clients due to tensor-level access. Our architecture, Re^{2}po, also falls under the category of reuse, but with a key difference: DNN model layers as the granularity rather than tensors.

In view of the methods mentioned above, model compression can reduce the model size but it may negatively impact accuracy while reusing models are sensitive to the granularity of the model. It is necessary to consider new aspects that are not addressed in previous work, such as splitting DNN models at the layer level and eliminating redundant layers. Therefore, we propose complete framework and implementation that address these new challenges.

TABLE I
Details of Selected Models

No.	Name	Size (MB)	Redundant Layers	Total Layers
1	EfficientNet-b0	21.5	49	360
2	EfficientNet-b3	49.5	77	574
3	EfficientNet-b6	173	720	986
4	MobileNet-v3-small	10.3	160	244
5	MobileNet-v3-large	17.7	213	312
6	MobileNet-v3-large-french	18.3	45	326
7	VGG16-bn	554	12	97
8	VGG16-bn-crnn	63.3	12	109
9	VGG16-bn-r	59.2	69	93

III. Analysis on Model Redudancy

In this section, several data analysis experiments are conducted to verify that there are significant redundant layers in models. We design an experiment to split the model layer by layer and record the relevant data. Concretely, we calculate hash values for the tensor data of each layer and then compare these values to identify redundant layers. To facilitate the description of the data collected in the experiment, we define model redundancy ratio as the percentage of redundant model layers relative to the total number of model layers.

To ensure the generalizability of the model data, we select different models of EfficientNet, MobileNet and VGG16 from publicly available repositories like Hugging Face for disassembly. Details of the selected models are shown in Table I and the complete experimental results are shown in Fig. 2. We find that each selected type of model contains a significant number of redundant layers, which cannot be overlooked. Three types of models in Fig. 2 all exhibit high redundancy rates. More specifically, models No. 3, No. 4, No. 5, and No. 9 have redundancy ratio of 73.02%, 65.57%, 68.27%, and 74.19%, respectively. Each of these models has a redundancy ratio exceeding 50%, indicating that more than half of the model layers are duplicated in storage and transmission.

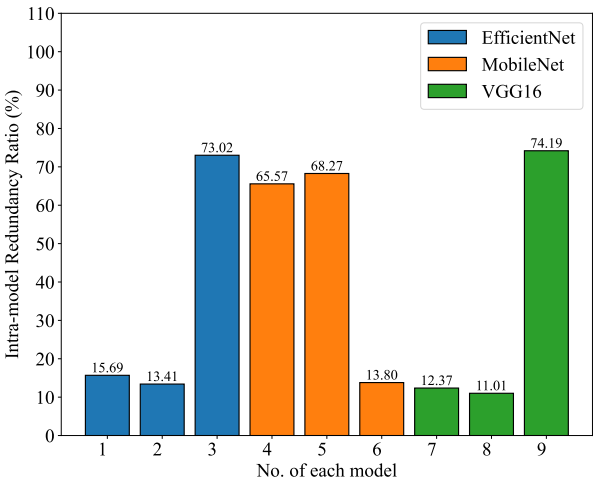


Fig. 2. Layer Redundancy Ratio of Different Models.

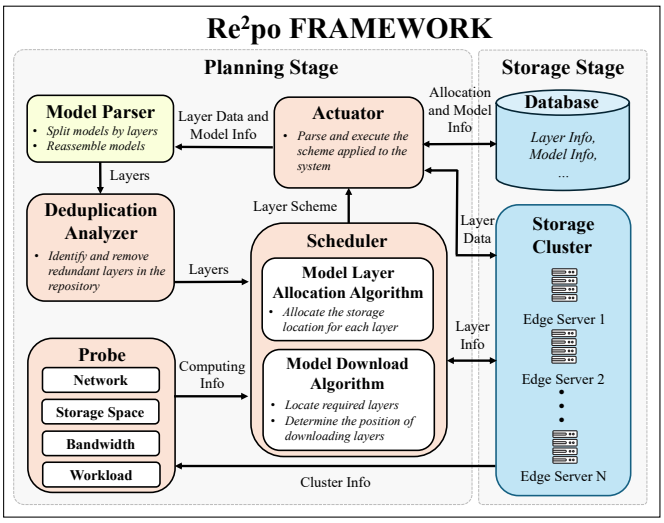


Fig. 3. An Overview of Re^{2}po Framework.

Further analysis of redundant layers shows that redundancy is mainly found in the bottom layers in image processing models such as EfficientNet and MobileNet. Bottom layers primarily handle foundational tasks in feature extraction, focusing on identifying basic features in input images. In contrast, redundancy in audio processing models like VGGish is concentrated in the embedding layers, which transform sparse and discrete inputs into dense and continuous vectors. Hence, the redundant transmission and storage of bottom layers and embedding layers result in resource wastage. Given their generality across models with similar function, these layers can be stored only once and reused by models.

In view of the analysis and findings above, we consider to deploy a distributed model repository at the edge, which can store models by layer, reducing redundant data storage and transmission time. This approach enhances storage efficiency and minimizes transmission latency. More precisely, the repository can quickly split and store models by layer, while also being able to rapidly reassemble and return the models when required.

IV. System Architecture

A. Re^{2}po Overview

In this section, we describe how Re^{2}po achieves layer-by-layer storage and eliminates redundant layers. To do this, we first analyze and introduce the information and functionality of each module within the framework. Following that, we explain the workflow of uploading a model to Re^{2}po and downloading a model from Re^{2}po, referred to as model uploading and model downloading, respectively.

Re^{2}po is a distributed redundancy-aware model repository framework. As illustrated in Fig. 3, Re^{2}po is composed of seven components: 1. **Model Parser:** Split the model into individual layers and reassemble it as needed. 2. **Deduplication Analyzer:** Identify and remove redundant layers to optimize the model structure. 3. **Probe:** Monitor system performance metrics, e.g., network bandwidth and storage utilization. 4. **Scheduler:** Gather performance data and model layer information to generate storage and download strategies. 5. **Actuator:** Execute the strategies formulated by the Scheduler to ensure efficient operations. 6. **Storage Cluster:** Store and manage the data associated with model layers. 7. **Database:** Maintain control information, including layer storage locations and model composition details.

In summary, Re^{2}po includes two stages, the planning stage and the storage stage. In the planning stage, the focus is on deciding storage locations and scheduling the download of layers to reassemble the model. Meanwhile, the storage stage is dedicated to storing the uploaded layers in the distributed model repository. The following is a detailed description of each component.

B. Planning Stage

Splitting models. Splitting models by layers and reassembling them into a complete model based on the received layer data and model information is the primary responsibility of the Model Parser. To be precise, Model Parser first parses the DNN model file, then abstracting out the information of each layer in the model for further exploration. Furthermore, Model Parser also places the model layers in the appropriate positions based on the queried model information and assembles them into a complete model. For the obtained model layers, Model Parser passes them to the Deduplication Analyzer.

Identifying redundancies. Identifying redundant layers by performing hash operations on the layer information of the entire model to obtain hash values and comparing them one by one is the primary function of the Deduplication Analyzer. In addition, Deduplication Analyzer searches the local repository's database to find any layers with the same hash value as the one being analyzed, marking layers with the same hash value as redundant and recording the redundant layer information. For redundant layers, the analyzer decides how many copies of each redundant layer to place based on the redundancy situation of each layer. Finally, it passes all the

data and information of the non-redundant layers, redundant layers, and copies of the redundant layers to the Scheduler.

Collecting infomation. Collecting information such as data, status, and other details from the storage cluster is the primary role of the Probe, which comprises four functions: *Network Detector, Bandwidth Detector, Storage Space Detector, and Workload Detector.* The *Network Detector* monitors and inspects all nodes in the storage cluster, ensuring efficient and secure network operation. By regularly sending and receiving survey packets, The Probe obtains and records the network information of the Storage Cluster, ensuring that healthy nodes can be quickly located when needed. The *Bandwidth Detector* periodically encapsulates survey packets and sequentially distributes them to each node in the storage cluster, recording the round-trip time of the packets. The reciprocal of this RTT is used to estimate network bandwidth. Since the data size of the survey packets is very small compared to the model layer data, their impact on network traffic is negligible. The *Storage Space Detector* can monitor the storage space of all nodes in the cluster in real-time. The *Workload Detector* monitors the load information of all nodes in the cluster, including the number of upload or download requests handled, and the CPU and memory usage of each edge server. All the information collected by the Probe is collectively referred to as computing information, which is passed to the Scheduler to assist in making more accurate schemes.

Implementing algorithms. Implementing the allocation and download algorithms is the primary responsibility of the Scheduler, enabling effective storage assignment and reassemble of model layers. The allocation algorithm assigns an appropriate storage location for each model layer. Similarly, the download algorithm is responsible for locating the required model layers and determining where to download them from. To minimize model download time and meet the storage space requirements, the allocation algorithm considers the overall performance of the cluster rather than isolated computing information factors. When a model needs to be stored, the Scheduler gathers information from components like Probe and Database to serve as the basis for decision-making. The Scheduler uses a heuristic algorithm to determine the placement strategy for each layer, providing redundancy for fault tolerance.

The workflow of the heuristic-base model layer storage location allocation algorithm is presented in Algorithm 1. At the outset, an empty placement set P is initialized to store the final placement of each layer. Then, each layer l is iterated over to find an appropriate storage location. Firstly, each model layer l is checked for redundancy in lines 3-5. The storage location $P[l]$ of the redundant layer is updated to align with the location of its primary instance $P[l_{\text{primary}}]$. In lines 6-7, we select nodes with high available storage resources and sufficient computing capacity as storage candidate nodes $Cand$. The lack of candidate nodes signifies an allocation failure in lines 8-10. Subsequently, the candidate nodes are traversed to select the most optimal node in terms of resources in lines 11-16. The storage location of layer l is then updated to the position of the optimal node, and the remaining storage capacity of the optimal node is adjusted accordingly in lines 17-18. Finally, after traversing all model layers, the complete placement set P is returned in line 20.

Algorithm 1: Heuristic-based Model Layer Storage Location Allocation Algorithm

Input: Set of nodes N, where each node $n \in N$ has a storage capacity $S[n]$ and a computing capacity $C[n]$; Set of layers $L = \{l_1, l_2, \ldots, l_m\}$, where each layer $l_j \in L$ has a size size(l_j) and an attribute indicating whether it is a redundant layer.

Output: Placement set $P = \{P[l_1], P[l_2], \ldots, P[l_n]\}$, where $P[l]$ denotes the placement of layer l. For any redundant layer l, $P[l]$ directly references the placement of its corresponding primary layer $P[l_{\text{primary}}]$.

1: Initialize $P = \{\}$;
2: **for** each layer $l \in L$ **do**
3: **if** l is a redundant layer **then**
4: $P[l] = P[l_{\text{primary}}]$; continue;
5: **end if**
6: max_storage $= \max(S[n] \mid n \in N$ and $C[n] > 0)$;
7: Cand $= \{n \in N \mid S[n] =$ max_storage and $C[n] > 0\}$;
8: **if** Cand $= \emptyset$ **then**
9: **return** False;
10: **end if**
11: best_node = None;
12: **for** each $n \in$ Cand **do**
13: **if** $S[n] - \text{size}(l) \geq 0$ **and** (best_node = None **or** $C[n] > C[\text{best_node}]$) **then**
14: best_node $= n$;
15: **end if**
16: **end for**
17: $P[l] =$ best_node;
18: $S[\text{best_node}] -= \text{size}(l)$;
19: **end for**
20: **return** P;

Algorithm 2 summarizes the layer download process. First, an empty set Q is initialized to store the locations of each required layer. Then, in line 2, each layer l specified in the model information M is iterated over. For each layer l, check its existence in the layer information L. If l exists, its location is stored in Q by referencing $L[l]$ in lines 3-4. If l does not exist in L, False is returned, indicating a missing layer location in lines 5-6. After traversing all model layers in M, the set Q, which contains the locations of all required layers for the model, is returned in line 9. The output Q provides efficient access to each layer's location, enabling subsequent processes to download layers as needed.

Executing strategies. Executing strategies involves translating the feasible layer scheme into concrete actions for storage and synchronization. After obtaining a feasible layer scheme, the Actuator is responsible for implementing it. Furthermore, it ensures synchronization between specific storage information within the cluster and the records stored in the Database.

Algorithm 2: Model Download Algorithm

Input: Dictionary of layers $L = \{l_1, l_2, \ldots, l_k\}$, where each key l_i represents a layer, and $L[l_i]$ denotes its storage location; Set of model information $M = \{l_1, l_2, \ldots, l_n\}$, where each element l_j is a layer required by the model.

Output: Model layer set $Q = \{Q[l_1], Q[l_2], \ldots, Q[l_n]\}$, where $Q[l]$ denotes the location of each required layer $l \in M$.

```
Initialize Q = {};
for each layer l ∈ M do
    if l ∈ L then
        Q[l] = L[l];
    else
        return False;
    end if
end for
return Q;
```

Moreover, it provides an execution queue to handle layer data with Storage Cluster concurrently, receiving execution requests from Scheduler.

C. Storage Stage

The Storage Stage primarily includes a Storage Cluster and a Database. The Storage Cluster provides storage locations for each model layer that requires storage, while the Database primarily holds metadata about the models, such as the layers included in each model and the node locations of these layers. In addition to model-related information, the Database also contains status information for the Storage Cluster, e.g., each server's storage space and workload data. The Storage Cluster mainly interacts with the Scheduler and Actuator to exchange model layer information and data, while the Database primarily supplies model information to the Actuator and records the actual storage scheme generated by the Actuator, including layer locations and other relevant details.

Overall, the main workflow of Re^{2}po includes model uploading which supports uploading models to the repository, and model downloading that handles downloading model from the repository. The specific workflow is shown in Fig. 4. Upon receiving a model upload request, Re^{2}po accepts a DNN model file produced by high-level frameworks like PyTorch as input. In the planning stage, the model is split into layers and redundant layers are removed. After scheduling, a storage scheme is generated and the model is stored accordingly. When receiving a model download request, Re^{2}po takes the DNN model information as input and locates the required layers, then reassembles the model to produce a complete DNN model. To sum up, the optimization goal of Re^{2}po is to improve storage resource utilization and maximize access efficiency by reducing model upload and download time.

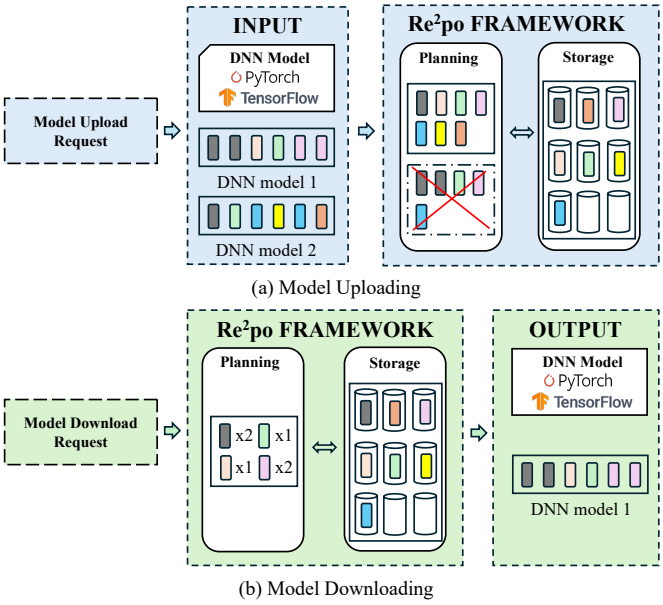


Fig. 4. The workflow of Model Uploading and Downloading.

TABLE II
DETAILED EXPERIMENTAL CONFIGURATIONS

Machine Type	Type 1	Type 2
Machine Quantity	2	2
CPU Device	13th Gen Intel(R) Core(TM) i9-13900K	ARMv8 Processor rev 1 (v8l)
Processor BaseFreq	5.80 GHz	2.201 GHz
CPU Threads	32	12
Memory Capacity	64GB	64GB
Operating System	Ubuntu 22.04.01	Tegra 5.10.104

V. IMPLEMENTATION AND EXPERIMENTS

In this section, we deploy a prototype system on 4 machines in a co-located LAN environment, to support the functionalities outlined in the Re^{2}po framework. The system enables model redundancy removal and distributed storage across nodes, as well as handling model upload and download requests. The detailed experimental configurations are shown in Table II.

A. Framework Implementation

Re^{2}po utilizes the open-source web framework FastAPI, the deep learning framework PyTorch, and the open-source distributed object storage service MinIO. Specifically, FastAPI is used to implement a gateway that handles model upload and download requests and manages distributed MinIO nodes.

PyTorch is employed to split models layer by layer, while MinIO is used to store the model layer data.

We modify the PyTorch framework to extract the identifiers and parameter information for each layer of the model. For each layer, a hash value is computed to uniquely identify different model layers. Re^{2}po determines whether a model layer is redundant by comparing hash values, which serves as the basis for storing layers in the model repository. To gather computational power information from the distributed nodes in the repository, we deploy probes on each edge node. These probes use Prometheus and Node-Exporter to conduct periodic and event-triggered tests to obtain status metrics for each node, such as storage capacity, workload, and network bandwidth. Furthermore, we use a dedicated time-series database to store all monitoring data.

The storage cluster primarily consist of edge servers with object storage systems. This storage method is able to efficiently process and store large amounts of unstructured model layer data, while also supporting dynamic scaling from small to large scale. Additionally, the simple RESTful API access pattern of object storage allows easy cross-platform access to model layer data, greatly simplifying data interaction in distributed computing environments. MinIO, a high-performance object storage solution compatible with Amazon S3 APIs, is deployed on these servers to provide basic storage services and simplify the storage process. After deploying MinIO, layer data can be stored on the corresponding servers within the cluster using MinIO's interfaces. Once Re^{2}po has collected all the computing information and model layer data, it uses storage allocation algorithms, based on greedy algorithm, to decide the storage locations for the model layers. After deciding, it invokes the upload interfaces of the MinIO services on the corresponding nodes to store the model layers.

The computing information of each node is equally crucial for downloading models. Re^{2}po uses this information to locate the storage positions of the required model layers and then calls the download interfaces of the MinIO services on the respective nodes to obtain the layer data. We also modify PyTorch's storage functionality. Instead of downloading the entire model directly, Re^{2}po reassembles the model into a complete form using control information.

B. Experiments and Analysis

Subsequently, we evaluate the performance of Re^{2}po framework and Chunk-Based Model Storage (CBMS) methods across various models and conduct comparative experiments to measure upload and download time [14] [15]. To ensure reliable results, each model is uploaded and downloaded ten times, with the duration recorded. The highest and lowest values are removed, and the average of the remaining data is calculated as the final result to minimize errors.

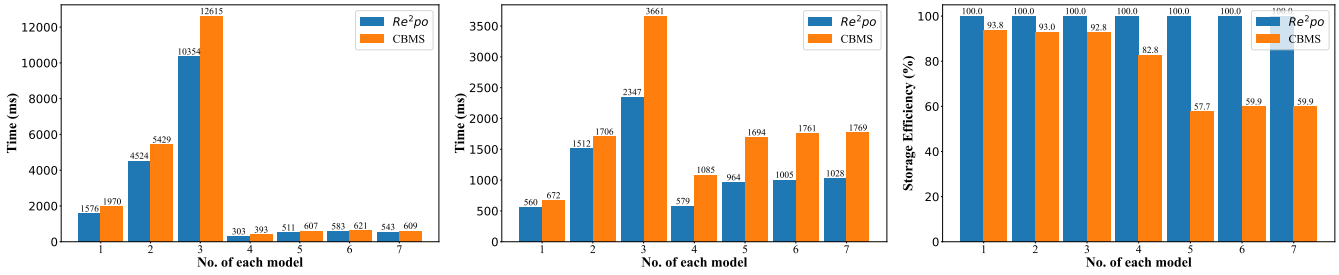
In our experiments, we measure the upload and download time while calculating the storage efficiency of different models, as shown in Fig. 5. Storage efficiency is defined as the ratio of effective model layers to the total number of model layers uploaded. Effective model layers refer to those without redundant data, while the total number of model layers uploaded refers to the number of model layers stored in the repository. We select several variants of similar models, which vary in size due to differences in parameter scales and computational costs (e.g., GPT-2) or in tasks and objectives (e.g., ResNet-50). Further details of the model information are shown in the Table III. For a model storage framework, the most important performance metrics are model access efficiency (i.e., the time required for model upload and download) and storage efficiency (i.e., the storage of effective data). Therefore, we design three experiments to compare the upload time, download time, and storage efficiency of Re^{2}po and CBMS based on access and storage efficiency.

Fig. 5(a) presents the comparative experimental results of the upload time for selected models. Re^{2}po demonstrates a consistent advantage over CBMS in upload time. For smaller models, such as the ResNet-50 series, Re^{2}po offers a slight edge over CBMS, with an improvement in upload time of around 40-100 ms. However, for larger models, such as those in the GPT-2 series, Re^{2}po yields a significant upload time improvement, reaching approximately 400-2000 ms. Analysis of redundant layers in these two types of models reveals that Re^{2}po's smaller upload time gains on ResNet-50 models, which are image processing models, are primarily due to their relatively small size and limited number of redundant layers. Conversely, GPT-2 models, being language models, contain more and larger redundant layers, which contributes to more substantial upload time improvements. Overall, Re^{2}po reduces the amount of data uploaded compared to CBMS. By calculating the average upload time improvement across the seven models, Re^{2}po reduces model upload time by 15.7% compared to CBMS.

The result of download time comparison is shown in Fig. 5(b), Re^{2}po shows a significant advantage. Re^{2}po can reduce download time by approximately 100-1300 ms. Similar to the upload time experiment, Re^{2}po's ability to reduce the amount of model layer data transferred leads to a shorter download time. By comparing larger models like the GPT-2 series with smaller models like the ResNet-50 series, we find that although both are affected by transfer volume, the upload time impact is generally less than that of download time. This difference arises because model upload and download, when deployed on a local object storage service, are primarily

TABLE III
DETAILS OF SELECTED MODELS

No.	Name	Size (MB)	Redundant Layers	Total Layers
1	GPT-2	548.1	11	160
2	GPT-2-medium	1536	23	316
3	GPT-2-large	3276.8	35	472
4	ResNet-50	102.5	56	320
5	ResNet-50-dc5	166.7	227	534
6	ResNet-50-panoptic	172.2	228	566
7	ResNet-50-dc5-panoptic	172.3	228	566



(a) Comparison of Upload Time Between Re^{2}po and CBMS. (b) Comparison of Download Time Between Re^{2}po and CBMS. (c) Comparison of Storage Efficiency Between Re^{2}po and CBMS.

Fig. 5. Comparison of Upload Time, Download Time and Storage Efficiency Between Re^{2}po and CBMS.

disk I/O operations. The machines used in our experiments have faster read speeds than write speeds, contributing to this variance. Calculating the average download time improvement across the seven models, Re^{2}po reduces average download time by 34.1% compared to CBMS.

The experimental result for storage efficiency is shown in Fig. 5(c). With Re^{2}po's mechanism for removing redundant layers, all model data stored effectively in the repository, meaning there is no duplicated data, achieving a storage efficiency of 100%. In contrast, CBMS stores the entire model in the repository, leading to redundant data storage, which reduces its storage efficiency. Calculating the average storage efficiency across the seven models, we find that Re^{2}po improves average storage efficiency by 26.2% compared to CBMS.

VI. Challenges and Opportunities

The distributed DNN model repository framework supports layer-by-layer model storage, offering the potential to increase storage efficiency while reducing model download time. Unlike chunk-based model storage method, the heterogeneity of Re^{2}po nodes and the diversity of model layers present unique challenges and complexities, necessitating timely and in-depth research tailored to practical applications. Our repository provides a new research approach for the application in the field of artificial intelligence.

1) Optimization of System Stability under High Concurrency: As a model repository, Re^{2}po typically needs to handle multiple user requests for downloading or uploading models. This means that during peak times, the repository must process tens of thousands of requests in a short period. Failure to handle these requests promptly is unacceptable for time-sensitive tasks. Even with load balancing monitoring across the nodes in the repository, if the load pressure cannot be alleviated in a timely manner, the overall system performance will still degrade. Therefore, quickly and effectively processing incoming requests is crucial for system stability.

The Re^{2}po employs optimization strategies such as message queues and service redundancy to ensure task completion quality and efficiency. By implementing service redundancy, multiple services can be initiated to accept user requests, which are then placed in a message queue in chronological order, awaiting processing. If necessary, task priorities can be set to ensure the completion of certain time-sensitive tasks. Service redundancy is optimized for service outages, allowing a healthy service to immediately replace a failed one, thus maintaining system stability. Considering these conditions, an efficient solution to handle high concurrency becomes the core solution to this issue.

2) Optimization of Model download and Service Deployment Time: When Re^{2}po handles model download requests, the process is typically separated from the DNN model service deployment nodes, resulting in significant model transmission latency. Additionally, in edge scenarios, many nodes possess both storage and computational capabilities. This architecture currently only leverages the storage capabilities of edge nodes, neglecting their computational potential, leading to resource wastage. Therefore, storage and computation should be considered together.

To reduce model transmission latency and effectively utilize the computational capabilities of edge nodes, the service deployment strategy must be adjusted. Utilize a serverless platform for model deployment and deploy this platform within the edge cluster. Transitioning from a service-centric to a data-centric approach can significantly reduce model transmission latency within the cluster and harness the computational capabilities of the edge nodes. However, this increases the system's complexity, making it crucial to balance computational and storage overhead. Consequently, designing an algorithm that effectively balances model storage and service computation is key to addressing this challenge.

3) Optimization of Redundant Layer Distribution in Re^{2}po Framework: The Re^{2}po framework determines the placement and number of replicas of redundant layers based on the redundancy count when handling upload requests. This static approach prevents the system from adjusting according to runtime conditions. Furthermore, the number of redundant layers does not correlate linearly with the likelihood of requests for these layers; while there is some correlation, it is not strong. This means user requests are unpredictable and constantly changing. Additionally, if the storage locations of different layers of a model are highly dispersed, the latency for downloading these layers during model reconstruction is relatively high. The number of layers and their storage locations significantly impact the overall performance of the system.

For nodes with high download request ratios, it indicates that the model layers stored on these nodes are frequently accessed, even if these layers are not redundant among models. For instance, if multiple users request the same DNN

model, and in an extreme case where the model does not redundant layers with any other models, the edge node storing these layers will face significant load pressure. Thus, creating replicas of highly requested non-redundant layers to distribute the user load is necessary. However, the storage capacity of edge nodes is limited, and the duration of high request ratios might be short-lived. Consequently, periodically deleting the replicas are created to alleviate load is essential. Therefore, developing a dynamic adjustment algorithm for the quantity and placement of highly accessed model layers and a periodic cleanup algorithm is crucial for enhancing system availability and reliability.

VII. CONCLUSION

We propose Re^{2}po, a redundancy-aware edge DNN model repository framework. Re^{2}po establishes a distributed model repository at the edge, intelligently removing redundant layers from uploaded models to optimize the storage and download process. When a model is needed, Re^{2}po can dynamically reassemble the required layers to reconstruct the complete model, thereby optimizing access speed and storage space. Experimental results show that compared to mainstream chunk-based model storage methods, Re^{2}po improves average upload time by 15.7%, download time by 34.1%, and storage efficiency by 26.2%.

Our work demonstrates that utilizing a redundancy-aware model repository in edge environments can effectively improve the storage and access efficiency of DNN models, thereby accelerating the serverless edge inference process. This provides new insights for deploying high-performance intelligent applications on resource-constrained edge devices. In the future, we plan to further optimize the performance of Re^{2}po and explore its application in a wider range of edge computing scenarios.

REFERENCES

[1] X. Li, X. Leng, and Y. Chen, "Securing serverless computing: Challenges, solutions, and opportunities," *IEEE Network*, vol. 37, no. 2, pp. 166–173, 2023.

[2] M. Rahimi Jafari, J. Su, Y. Zhang, O. Wang, and W. Zhang, "Pisel: Pipelining dnn inference for serverless computing," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, 2024, pp. 1951–1960.

[3] A. Ali, R. Pinciroli, F. Yan, and E. Smirni, "Optimizing inference serving on serverless platforms," *Proceedings of the VLDB Endowment*, vol. 15, no. 10, 2022.

[4] T. Rausch, A. Rashed, and S. Dustdar, "Optimized container scheduling for data-intensive serverless edge computing," *Future Generation Computer Systems*, vol. 114, pp. 259–271, 2021.

[5] S. Park, J. Choi, and K. Lee, "All-you-can-inference: serverless dnn model inference suite," in *Proceedings of the Eighth International Workshop on Serverless Computing*, 2022, pp. 1–6.

[6] A. Fuerst and P. Sharma, "Faascache: keeping serverless computing alive with greedy-dual caching," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 386–400.

[7] F. Romero, G. I. Chaudhry, Í. Goiri, P. Gopa, P. Batum, N. J. Yadwadkar, R. Fonseca, C. Kozyrakis, and R. Bianchini, "Faa $ T: A transparent auto-scaling cache for serverless applications," in *Proceedings of the ACM symposium on cloud computing*, 2021, pp. 122–137.

[8] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, "Pruning and quantization for deep neural network acceleration: A survey," *Neurocomputing*, vol. 461, pp. 370–403, 2021.

[9] J. Kim, S. Chang, and N. Kwak, "Pqk: model compression via pruning, quantization, and knowledge distillation," *arXiv preprint arXiv:2106.14681*, 2021.

[10] J. Xue, Y. Zhao, S. Huang, W. Liao, J. C.-W. Chan, and S. G. Kong, "Multilayer sparsity-based tensor decomposition for low-rank tensor completion," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 11, pp. 6916–6930, 2021.

[11] M. Wang and Q. Zhang, "Optimized data storage algorithm of iot based on cloud computing in distributed system," *Computer Communications*, vol. 157, pp. 124–131, 2020.

[12] S. Li and T. Lan, "Hotdedup: Managing hot data storage at network edge through optimal distributed deduplication," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 247–256.

[13] M. Madhyastha, R. Underwood, R. Burns, and B. Nicolae, "Dstore: A lightweight scalable learning model repository with fine-grain tensor-level access," in *Proceedings of the 37th International Conference on Supercomputing*, 2023, pp. 133–143.

[14] L. Conde-Canencia and B. Hamoum, "Deduplication algorithms and models for efficient data storage," in *2020 24th International Conference on Circuits, Systems, Communications and Computers (CSCC)*. IEEE, 2020, pp. 23–28.

[15] S. S. Saraswathi and N. Malarvizhi, "Distributed deduplication with fingerprint index management model for big data storage in the cloud," *Evolutionary Intelligence*, vol. 14, pp. 683–690, 2021.